

Android Camera 倒车影像功能介绍

文档版本
发布日期

V1.0
2024-05-17

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档主要介绍 Android 平台 Camera 倒车影像的相关知识，内容涵盖倒车影像方案、倒车影像架构、Nativecamera 倒车、Nativecamera 快速倒车、Kernel 倒车、LK 倒车及典型案例分析。

读者对象

本文档主要适用于 Android 平台的用户。

缩略语

缩略语	英文全名	中文解释
ACC	Adaptive Cruise Control	自适应巡航控制
CVBS	Composite Video Broadcast Signal	复合同步视频广播信号
GPIO	General Purpose Input Output	通用输入/输出口
LK	Little Kernel	微型内核
PAG	Parking Assist Guidelines	倒车辅助线
RVC	Rear View Camera	后视影像系统

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.0	2024-05-17	第一次正式发布。

关键字

Camera、倒车影像、RVC、Nativecamera、Kcamera、LK Camera

目 录

1 倒车影像方案.....	1
2 倒车影像架构.....	2
3 Nativecamera 倒车	4
3.1 倒车 GPIO 模块.....	4
3.2 倒车检测及信号处理	5
3.3 倒车引导线	7
4 Nativecamera 快速倒车	9
5 Kernel 倒车.....	13
6 LK 倒车	15
7 TP9950 MIPI CLK inactive 典型案例分析	17

图目录

图 2-1 Camera 倒车影像框架	2
图 2-2 Camera 倒车影像硬件流程	3
图 3-1 RVC GPIO 倒车流程	4
图 3-2 RVC GPIO 中断 handle	4
图 3-3 Nativecamera 检测 RVC_STATE	5
图 3-4 Nativecamera 检测 RVC_MODE 进入 RVC 倒车流程	5
图 3-5 奇偶合并	6
图 3-6 Camera 数据整合	6
图 3-7 无引导线（左）、有距离引导线（中）、动态引导线（右）	7
图 3-8 使用 SurfaceFlinger 画引导线	8
图 4-1 Nativecamera 启动流程	9
图 4-2 关闭 SELinux	10
图 4-3 Camera 驱动从 on post-fs 提前到 on fs	11
图 4-4 Nativecamera 及依赖项从 on boot 阶段提前到 on post-fs	11
图 4-5 将 vendor.power-hal-4-0-sprd 进程从 init 的 on boot 阶段提前到 on post-fs	11
图 4-6 在 init.rc 中直接打开倒车服务	12
图 4-7 关闭 Nativecamera 中的检测标志位倒车	12
图 5-1 Kcamera 简要框图	13
图 5-2 开启 CONFIG_KCAMERA_FEATTRUE 宏	14
图 5-3 检测 RVC 信号开启 Kernel 倒车	14
图 6-1 LK Camera 架构	15
图 6-2 LK Camera 启动流程	16
图 7-1 TP9950 LP 切 HS 的时序	17

表目录

表 1-1 倒车影像方案比较	1
表 3-1 Camera 数据整合流程用到的主要接口	6
表 4-1 优化后启动时间分解	10

1 倒车影像方案

目前倒车影像方案有 App 应用倒车、Nativecamera 倒车、Nativecamera 快速倒车、Kernel 倒车和 LK (Little Kernel, 微型内核) 倒车, 方案优缺点及适用范围见[表 1-1](#)。

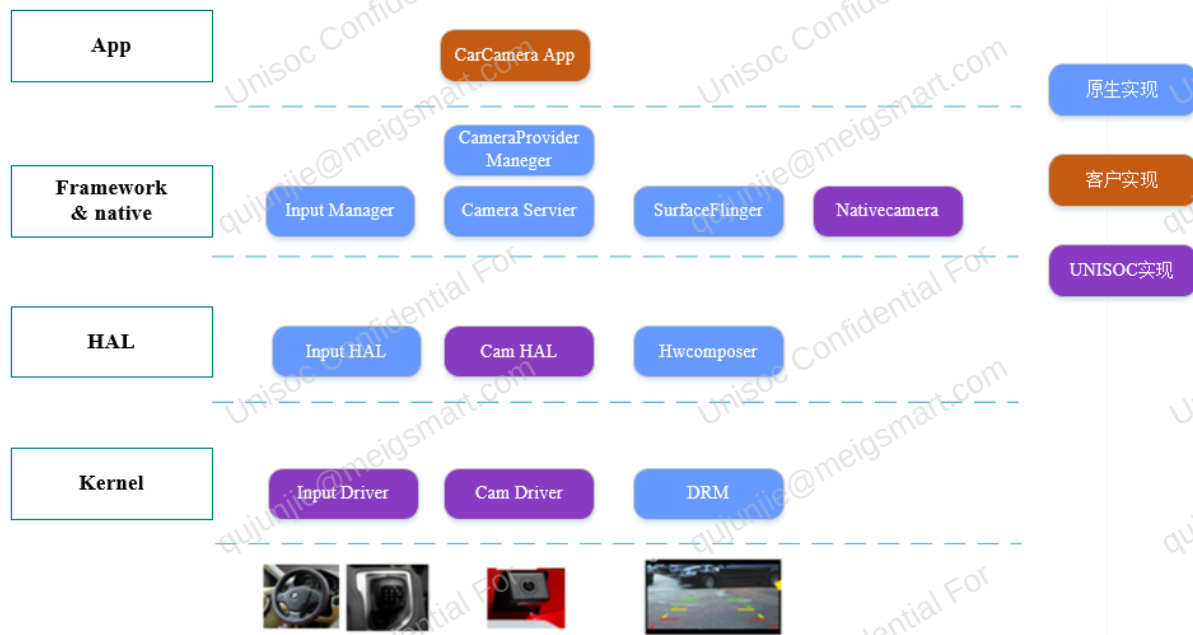
表1-1 倒车影像方案比较

倒车影像方案	优点	缺点	适用范围
App 应用倒车	可以使用 Android 多种渲染应用, 功能丰富	冷启动时间长, Android 启动之后才能开启倒车影像	Android 10、Android 12、Android 13 的车机项目 (客制化实现, 不同项目需要针对性适配)
Nativecamera 倒车	直接下发给 HAL 层, 可以使用 SurfaceFlinger 画图, 耗时少于 App 应用倒车方案	依赖于 Android, 需要 Android 相关进程辅助	Android 10、Android 12、Android 13 的车机项目 (不同项目可能需要针对性适配)
Nativecamera 快速倒车	冷启动时间优化到 6.5s 左右	依赖于 Android, 需要 Android 相关进程辅助	已集成在 Android 13 UIS7870SC 项目 (其他项目需要调试适配)
Kernel 倒车	冷启动时间不超过 4s	刚启动时缺少画线功能、不支持 CVBS (Composite Video Broadcast Signal, 复合同步视频广播信号) 等	已集成在 Android 10 后装基线的 rls1 项目分支
LK 倒车	冷启动时间低于 3s	缺少 GPU 加速等	已集成在 Android 13 UIS7870 前装项目的 automotive 分支

2 倒车影像架构

Camera 倒车影像架构如图 2-1，Camera 倒车信号会被 App 监听，App 下发倒车指令。

图2-1 Camera 倒车影像框架

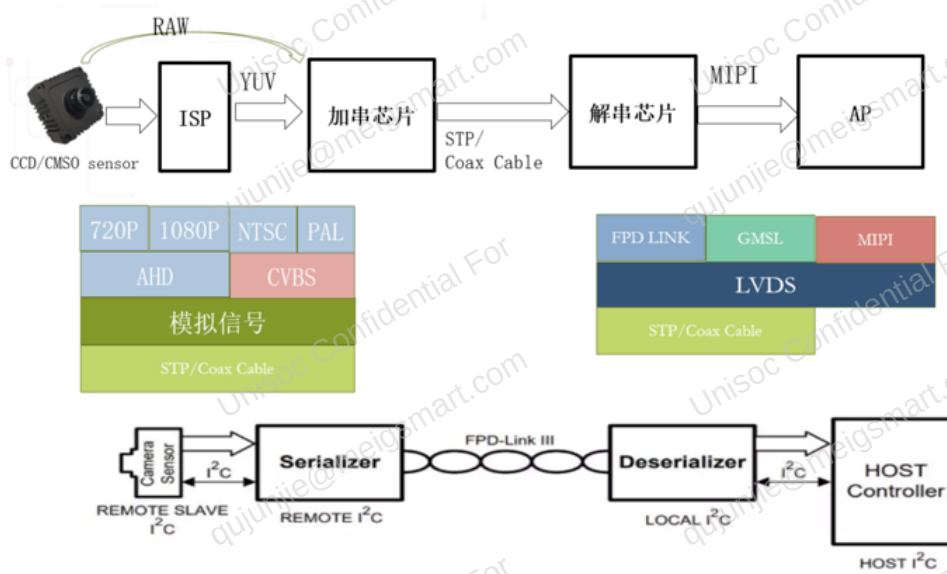


倒车影像包括下面三要素：

- 倒车信号：SoC 通过检测 GPIO（General Purpose Input Output，通用输入/输出）信号，来传输倒车信号给 App，开启倒车应用。
- 倒车 sensor：传输倒车影像给 SoC，然后由图像处理系统来处理图像信息。
- 倒车画线：展锐平台根据倒车图像进行画线。

Camera 倒车影像需要的硬件模块主要有 sensor、转接芯片、SoC 芯片、MIPI、I2C 和 GPIO 等，由于车载上 SoC 芯片接收的是 YUV 数据，通过 DCAM LITE 透传，或 DCAM 格式转换、旋转和镜像，在 ISP 缩放送给上层。

图2-2 Camera 倒车影像硬件流程

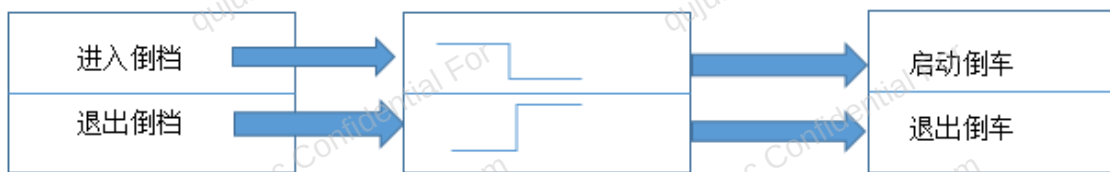


3 Nativecamera 倒车

3.1 倒车 GPIO 模块

当汽车进入或者退出倒档模式后，会通过电平拉高或拉低的方式，触发信号给 GPIO 模块，再通知系统启动或者退出倒车应用 App。

图3-1 RVC GPIO 倒车流程



1. GPIO 检测代码是 `unisoc-gpio-rvc.c`，通过 `unisoc_gpio_rvc_handle` 来处理中断，记录 `RVC_STATE`。

图3-2 RVC GPIO 中断 handle

```
static irqreturn_t unisoc_gpio_rvc_handler(int irq, void *dev_id)
{
    int value;
    struct unisoc_gpio_rvc *gpio_rvc = (struct unisoc_gpio_rvc *)dev_id;

    value = !!gpio_get_value(gpio_rvc->gpio);
    dev_info(gpio_rvc->dev, "%s called gpio=%d value=%d gpio_rvc->input_dev=0x%p \n",
__func__, gpio_rvc->gpio, value, gpio_rvc->input_dev);
    irq_set_irq_type(irq, value ? IRQ_TYPE_LEVEL_LOW : IRQ_TYPE_LEVEL_HIGH);
    gpio_rvc->rvc_status = value;
    // wake_lock_timeout(&system_wakeup_lock, msecs_to_jiffies(3000));
    if (gpio_rvc->rvc_status == RVC_STATE_ON) {
        dev_info(gpio_rvc->dev, "rvc on\n");
    }

#ifdef SPRD_KCAMERA
    rvc_onoff = RVC_STATE_ON;
#endif
}
```

2. Nativecamera 通过读取 `RVC_STATE` 来打开 Camera 倒车。

图3-3 Nativecamera 检测 RVC_STATE

```
int RVC_Start(void)
{
    int ret;
    pthread_t tid_event_manager;
    pthread_t tid_service_manager;

    ret = RVC_Create_EventsManager_Thread(&tid_event_manager);
    if (ret) {
        ALOGI("create RVC EventsManager Thread faild");
        return -1;
    }

    void *RVC_EventsManager_Thread(void *p_data)
    {
        p_data = NULL;

        int fd_keys;
        struct input_event event;

        // open input device
        fd_keys = open(INPUT_DEVICE, O_RDONLY | O_CLOEXEC);
        if (fd_keys < 0) {
            E("could not open %s, %d", INPUT_DEVICE, errno);
            return NULL;
        }

#ifdef SPRD_KCAMERA
        // detect default gpio level of rvc
        if (RVC_Get_Gpio_Value() > 0) {
            ALOGI("Enter RVC by default gpio level");
            g_rvc_current_events = START_NATIVECAMERA;
        }
#endif
    }
}
```

3.2 倒车检测及信号处理

以 Android 13 为例，Nativecamera 倒车会检测 RVC_MODE，如果条件满足才会进入到 RVC（Rear View Camera，后视影像系统）倒车流程。

图3-4 Nativecamera 检测 RVC_MODE 进入 RVC 倒车流程

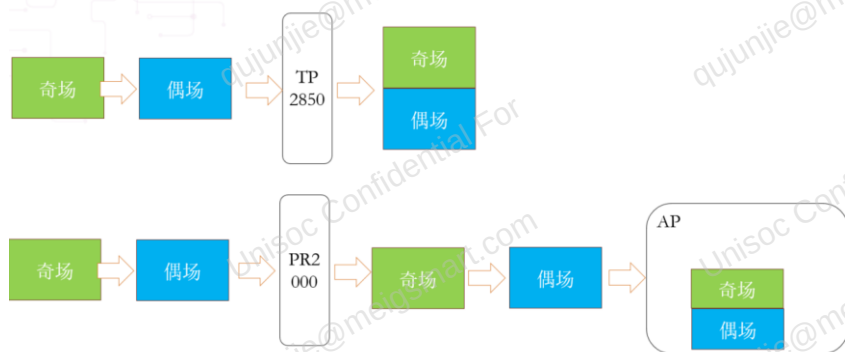
```
int main(int argc, char *argv[])
{
    if(RVC_MODE==1){
        RVC_Start();
        return 0;
    }

    if(TEST_MODE==1){
        // User test mode
        if (argc < 6) {
            printf("%s id width height rotate mirror. eg:nativecamera 0 1280 720 90\n", argv[0]);
            return 0;
        }
        int status = 0;
        NativeCameraHidl native_camera;

        int g_camera_id = atoi(argv[1]);
        int g_sensor_width = atoi(argv[2]);
        int g_sensor_height = atoi(argv[3]);
        int g_rotate = atoi(argv[4]);
        int mirror = atoi(argv[5]);
    }
}
```

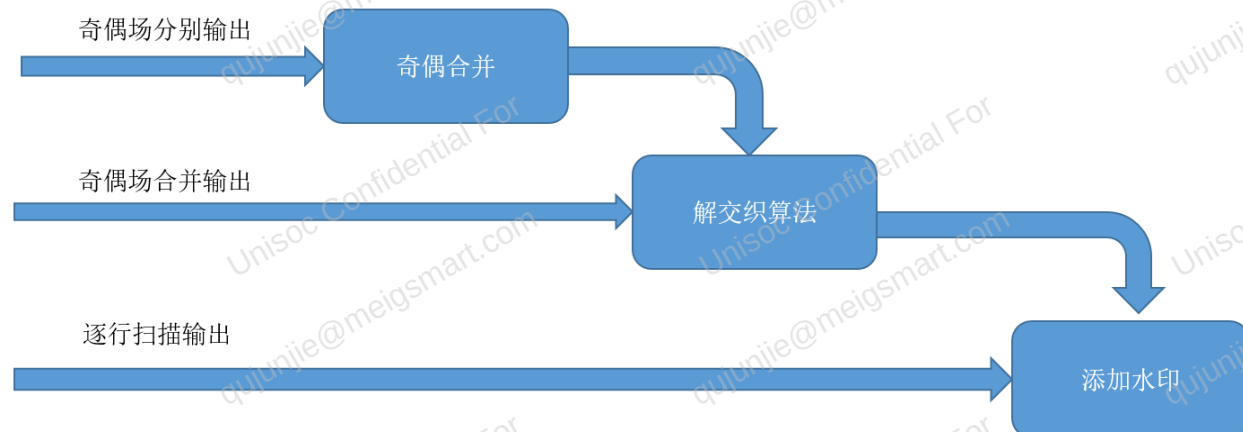
Nativecamera 开启倒车流程下发完毕后 Camera 即可出帧，但不同制式 Camera 出帧方式不一样。对于 CVBS 制式 Camera 需要进行解交织，例如 TP2850 会输出奇偶合帧；而 PR2000 单独输出奇帧和偶帧，需要展锐平台进行奇偶合并。

图3-5 奇偶合并



当前解交织算法是 libdeinterlace_sw_sprd.so，在 HAL 层进行处理。而且可以在 HAL 层添加水印，Camera 数据整合流程见图 3-6。

图3-6 Camera 数据整合



Camera 数据整合流程用到的主要接口如表 3-1（奇偶合并目前基线未支持）：

表3-1 Camera 数据整合流程用到的主要接口

API	参数	说明
processing_odd_even_frame	struct prev_handle *handle, cmr_u32 camera_id, cmr_u32 frm_id, struct frm_info *info	解交织处理

API	参数	说明
int ToAddWaterMarkInPic	long virtAddr, uint32_t w, uint32_t h	添加水印

3.3 倒车引导线

倒车引导线通常称为 PAG（Parking Assist Guidelines，倒车辅助线），目前，倒车视频可分为三种：

- 只有画面，无距离参考线，如图 3-7（左）。
- 画面中有静态的距离参考线，如图 3-7（中）。
- 画面中有静态的距离参考线，并且带有动态倒车轨迹线，如图 3-7（右），目前平台暂不支持动态引导线。

图3-7 无引导线（左）、有距离引导线（中）、动态引导线（右）



以 Android 13 车载项目上 Nativecamera 画线功能为例，使用 drawline.c 中 SurfaceFlinger 画引导线，其他平台使用逻辑有所差异。

图3-8 使用 SurfaceFlinger 画引导线

```
void *drawline_thread(void* args)
{
    connectSurfaceFlinger();
    ANativeWindow *surface = getSurface().get();
    connectBufferQueue(surface);
    setBufferTransaction();
    setBuffer(surface);

    int fenceFD = -1;
    int rvexitflag = 0;
    err = surface->dequeueBuffer(surface, &mNativeBuffer, &fenceFD);

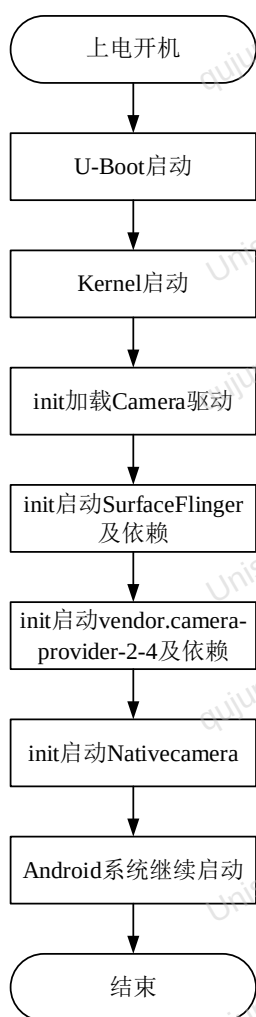
    if (err != NO_ERROR) {
        ALOGE("dequeueBuffer err....");
    }
    sp<Fence> fence(new Fence(fenceFD));
    int waitResult = fence->waitForever("dequeueBuffer_EmptyNative");
    if (waitResult != OK) {
        ALOGE("Fence wait err....");
    }
    sp<GraphicBuffer> buff(GraphicBuffer::from(mNativeBuffer));
    unsigned char *data = new unsigned char[mWidth*4*mHeight]; root system
```

4 Nativecamera 快速倒车

原始 Nativecamera 倒车功能启动时间 12.5s 左右，倒车出图时间较长，Nativecamera 快速倒车即在原有的基础上优化倒车时间。

Nativecamera 上电启动流程如图 4-1，主要从流程中进行优化，U-Boot 和 init 两个阶段为主要优化方向（本章 Nativecamera 快速倒车以 Android 10 系统的 UIS7862A 为例）。

图4-1 Nativecamera 启动流程



目前 U-Boot 阶段优化 2.3s，init 阶段优化 3.7s，总体优化了 6s。从系统开机到 Nativecamera 倒车影像出图，时间从 12.5s 优化到 6.5s。优化后启动时间分解见表 4-1，具体优化内容将依次介绍。

表4-1 优化后启动时间分解

启动阶段	优化前耗时	优化后耗时	优化时间	优化说明
PBINT	0.7s	0.7s	无	无
U-Boot	4.3s	2.0s	2.3s	U-Boot 功能裁剪和关闭 secure boot
Kernel > init	0.9s	0.9s	无	无
init > Camera 驱动加载	2.4s	1.2s	2.2s	Camera 驱动从 init on post-fs 提前到 on fs 加载
Camera 驱动加载> Nativecamera 启动	1.2s	0.2s		Nativecamera 及依赖进程从 on boot 提前到 on post-fs 启动
Nativecamera 启动> Camera 出图	3s	1.5s	1.5s	- 裁剪 sensor_config.xml 未使用的 Camera，节省 Camera 遍历和初始化时间 0.5s - vendor.power-hal-4-0-sprd 从 init on boot 提前到 on post-fs 启动，节省 initPowerHint 等待时间 1s
Total	12.5s	6.5s	6s	总优化时间 6s

1. 关闭 SELinux 功能，通过移除 secure boot 功能来缩短倒车影像相关功能在申请权限上的时间。

图4-2 关闭 SELinux

```
diff --git a/init/selinux.cpp b/init/selinux.cpp
index cd17fb5..a90c6de 100644
--- a/init/selinux.cpp
+++ b/init/selinux.cpp
@@ -98,6 +98,7 @@
bool IsEnforcing() {
{
+   return false;
   int fd(open("/mboot/selinux", O_RDONLY | O_CLOEXEC | O_BINARY));
   if (fd != -1) {
       char v = 0xff;
diff --git a/rootdir/init.rc b/rootdir/init.rc
```

2. 优化 init.rc 中 Camera 的 ko 模块上电时间，将 Camera 驱动加载从 on post-fs 阶段提前到 on fs 阶段。

图4-3 Camera 驱动从 on post-fs 提前到 on fs

```
diff --git a/kernel_module/camera.rc b/kernel_module/camera.rc
index 5e89e04..1fb3782 100644
--- a/kernel_module/camera.rc
+++ b/kernel_module/camera.rc
@@ -1,4 +1,4 @@
-on post-fs
+on fs
insmod ${ro.vendor.ko.mount.point}/socko/sprd_sensor.ko
insmod ${ro.vendor.ko.mount.point}/socko/sprd_flash_drv.ko
insmod ${ro.vendor.ko.mount.point}/socko/sprd_camera.ko
```

3. 将 Nativecamera 及依赖的 SurfaceFlinger 和 vendor.camera-provider-2-4 等进程从 on boot 阶段提前到 on post-fs 阶段。

图4-4 Nativecamera 及依赖项从 on boot 阶段提前到 on post-fs

```
diff --git a/rootdir/init.rc b/rootdir/init.rc
index 0f5f70c..96e592f 100644
--- a/rootdir/init.rc
+++ b/rootdir/init.rc
@@ -405,6+405,17@@on post-fs
mkdir /metadata/apex 0700 root system
mkdir /metadata/apex/sessions 0700 root system

+start surfaceflinger
+start hidl_memory
+start vendor.configstore-hal
+start vendor.gralloc-2-0
+start vendor.hwcomposer-2-1
+start vendor.camera-provider-2-4
+start nativecamera
```

4. Nativecamera 启动过程中在初始化 initPowerHint()时耗时 1s, 原因是 initPowerHint 需要访问 IPower 接口, 但负责注册 IPower 接口的 vendor.power-hal-4-0-sprd 进程启动较晚, 从而造成 Nativecamera 等待耗时。将 vendor.power-hal-4-0-sprd 进程从 init 的 on boot 阶段提前到 on post-fs 阶段启动。

图4-5 将 vendor.power-hal-4-0-sprd 进程从 init 的 on boot 阶段提前到 on post-fs

```
diff --git a/rootdir/init.rc b/rootdir/init.rc
index 0f5f70c..96e592f 100644
--- a/rootdir/init.rc
+++ b/rootdir/init.rc
@@ -405,6+405,17@@on post-fs
mkdir /metadata/apex 0700 root system
mkdir /metadata/apex/sessions 0700 root system

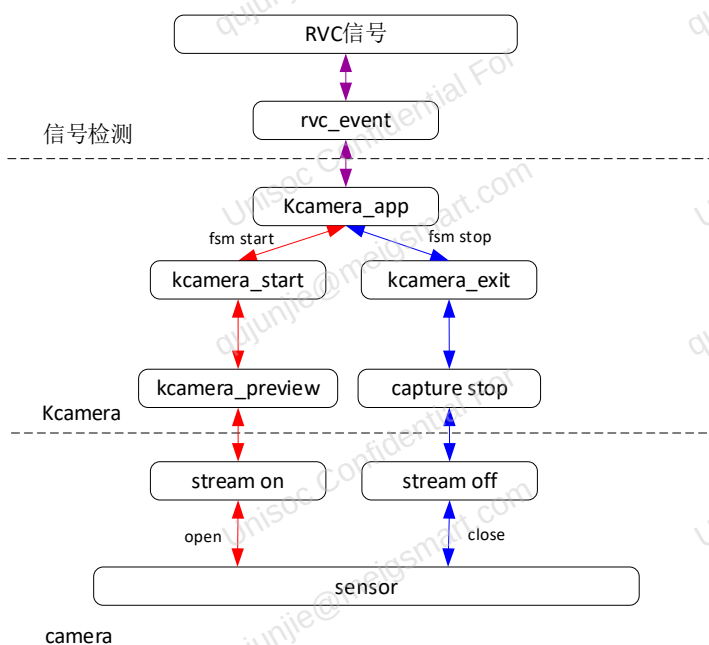
+start surfaceflinger
+start hidl_memory
+start vendor.configstore-hal
+start vendor.gralloc-2-0
+start vendor.hwcomposer-2-1
+start vendor.camera-provider-2-4
+start vendor.power-hal-4-0-sprd
+start nativecamera
on late-fs
```


5 Kernel 倒车

Kernel 倒车即在 Kernel 里实现倒车。Nativecamera 即便优化了倒车时间，最短也需要 6.5s 左右时间，有时仍无法满足用户期望。由于对倒车影像高实时性的要求，Kernel 倒车功能便应时而生，让倒车信号发起到倒车影像出现时间继续缩短。

图 5-1 是 Kernel 倒车的简要框图，通过 GPIO 信号检测开启 Camera，仅在 Android 10 车载项目上支持。Kernel 倒车将用户空间 oem 层代码适配到 Kernel 空间，并在 Kernel 空间运行 kcamera_app 线程启动 Camera。

图5-1 Kcamera 简要框图



Kernel 倒车需要开启 CONFIG_KCAMERA_FEATTRUE 宏，然后通过检测 RVC 或 ACC（Adaptive Cruise Control，自适应巡航控制）硬件信号触发倒车影像功能。

图5-2 开启 CONFIG_KCAMERA_FEATURE 宏

```
device/sprd/shark13/项目名/项目名_Base.mk
#unisoc license support
HALO_SUPPORT :=true

#enable CONFIG_KCAMERA_FEATURE
CONFIG_KCAMERA_FEATURE := true

bsp/device/shark13/androidq/项目名/项目名_base/common.cfg
export CONFIG_IVI_FEATURE="true"

# for kcamera feature
export CONFIG_KCAMERA_FEATURE="true"
```

sensor 作为 App 和 HAL 层的交互部分放进 Kernel 里，直接使用 kcamera_app.c 代替原先的 App 控制，代码路径在 bsp/kernel/kernel4.14/drivers/misc/sprd_cam/core/isp2.6/kcamera。

在 HAL 层适配硬件 sensor 扫描，去除 auto detect 功能，通过检测 RVC 信号开启倒车影像。

图5-3 检测 RVC 信号开启 Kernel 倒车

```
.void RVC_Update_Current_Event(struct input_event event)
{
    char prop[PROPERTY_VALUE_MAX];
    @@-1841,16-+1864,16@@void RVC_Update_Current_Event(struct input_event event)
    // update current event
    property_get(BOOT_PROP, prop, "null");
    if (event.code == CODE_RVC_ON && event.value == 1) { /* start rvc */
        if (!strcmp(prop, BOOT_FLAG)) {
            if (RVC_Get_Boot_value() == BOOT_COMPLETED) {
                g_rvc_current_events = START_APK;
            } else {
                g_rvc_current_events = START_NATIVECAMERA;
            }
            //g_rvc_current_events = START_NATIVECAMERA;
        }
    } else if (event.code == CODE_RVC_OFF && event.value == 1) { /* exit rvc */
        if (g_rvc_current_events == START_APK) {
            g_rvc_current_events = EXIT_APK;
        } else if (g_rvc_current_events == START_NATIVECAMERA) {
            g_rvc_current_events = EXIT_NATIVECAMERA;
        }
        //g_rvc_current_events = EXIT_NATIVECAMERA;
    }
}
```

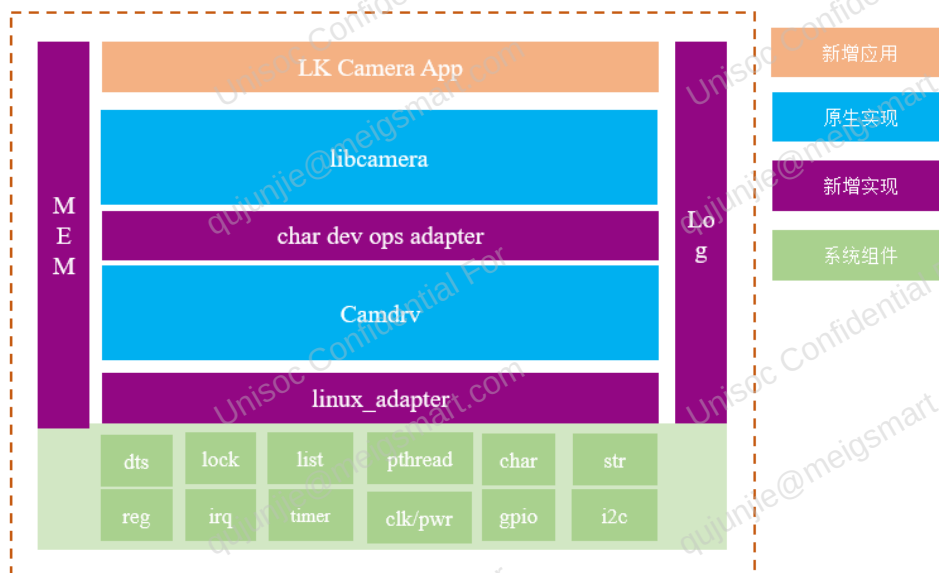
6 LK 倒车

车载的一个重要特性是冷启动开机阶段能快速启动倒车应用。为了使 Camera 能提前加载来响应快启的需求，Camera 的驱动和应用也放在 LK 阶段。目前 Android 13 前装项目 UIS7870 上集成了该功能，需要开启 SPRD_CAMERA_LK 宏（位于 bsp/bootloader/lk/project/项目名.mk）。

当前 LK Camera 支持如下特性：

- YUV sensor（目前固定为 gc2145 sensor 和 ipg 模式）
- DCAM LITE 透传、DCAM/ISP offline
- LK 倒车显示
- GPIO 模拟倒车事件

图6-1 LK Camera 架构



Android 启动后，RVC service 会接管倒车的状态，并决定当前是否需要关闭 LK RVC，或者根据当前的启动阶段来启动对应的 Camera 应用来服务于 RVC 场景。

说明

LK RVC 和 Android RVC service 通过 LK Comm 模块直接通信。

bsp/bootloader/lk/app/lk_camera/start_camera.c 为 LK 的 Camera demo，LK 启动后会有一个 core 给 Camera 使用，线程启动后会调用 Camera 的接口 run_camera。当前逻辑是 LK 启动后会启动 Camera 出图，每一帧会去判断是否有倒车信号，若有倒车触发，则调用显示接口，立即显示。可自行更改启动逻辑，如检测到有信号后再启动 Camera。

外部接口包含：

- rvc_gpio_check(): 检测是否有倒车信号
- reserve_flip: 显示 Camera

RVC 倒车启动流程分为 4 个阶段：

1. LK 启动过程中会启动 LK Camera，监听是否有倒车信号以及 Android 端发来的 Camera 命令。
2. 倒车信号接入后，立即显示倒车画面。
3. Android 启动过程会启动 RVC service，用于获取 LK Camera 状态。若处于 running，则等待倒车信号关闭，同时 LK Camera 退出，随后 Android RVC 监听倒车信号。
4. 当第二次及后续倒车信号到来后，都由 Android RVC 来启动 Camera 应用或 Nativecamera 应用。

图6-2 LK Camera 启动流程



7

TP9950 MIPI CLK inactive 典型案例分析

问题现象

SL8541E 上调试一颗 AHD 转 MIPI 的 Camera 转接芯片 TP9950，DCAM 收不到数据。

分析思路

软件读 CSI 的 phy state(offset 0x18)状态是 0x20000，CLK 处于 inactive。需要量取硬件波形进行分析。

根本原因

量取硬件波形发现 MIPI CLK 从 LP 切 HS 的时序中没有 Zero 区域，可能会导致 phy 的 CLK 是 inactive 状态。TP9950 厂商确认输出的 MIPI CLK 无法调整，故需要在 SL8541E 的 MIPI 进行调整。

图7-1 TP9950 LP 切 HS 的时序



解决方案

由于 TP9950 的波形非标准，输出的 CLK 波形缺少 HS-0 波形，接收端无法正确检测到 MIPI CLK 时序，从而无法进入 HS 模式，CLK lane 也无法激活。

需要在 CSI phy 初始化时配置寄存器跳过 MIPI CLK 时序 HS-0 波形的检测。CLK lane 没有 HS-0，所以只能等待一定的时间开始接收高速数据；如果有 HS-0，可以自动判断。

文件：bsp/modules/camera/sensor/csi2/sprd/receiver_r2p0/csi_driver.c、csi_phy_init 寄存器配置如下（需根据具体项目选择对应路径添加，以 SL8541E 平台为例）：

```

#define MIPI_CLK_NO_HS_ZERO
void csi_phy_init(struct csi_dt_node_info *dt_info, int32_t idx)
{
    struct csi_phy_info *phy = NULL;
    if (!dt_info) {
        pr_err("fail to get valid phy ptr\n");
        return;
    }
    @@ -1068,6 +1071,21 @@ void csi_phy_init(struct csi_dt_node_info *dt_info, int32_t idx)
        phy_write(idx, 0x4d, 0x48, 0xff);
        phy_write(idx, 0x5d, 0x68, 0xff);
        phy_csi_path_cfg(dt_info, idx);
    #ifdef MIPI_CLK_NO_HS_ZERO
+    phy_write(idx, 0x04, 0x50, 0xff);
+    phy_write(idx, 0x04, 0x50, 0xff);
+    phy_write(idx, 0x37, 0x06, 0xff);
+    phy_write(idx, 0x37, 0x06, 0xff);
+    phy_write(idx, 0x47, 0x06, 0xff);
+    phy_write(idx, 0x47, 0x06, 0xff);
+    phy_write(idx, 0x57, 0x06, 0xff);
+    phy_write(idx, 0x57, 0x06, 0xff);
+    phy_write(idx, 0x67, 0x06, 0xff);
+    phy_write(idx, 0x67, 0x06, 0xff);
+    phy_write(idx, 0x77, 0x06, 0xff);
+    phy_write(idx, 0x77, 0x06, 0xff);
    #endif
        break;
    case PHY_2P2_M:
        /* 2p2lane phy as a 2lane M phy */
    @@ -1087,6 +1105,8 @@ void csi_phy_init(struct csi_dt_node_info *dt_info, int32_t idx)
        pr_err("fail to get valid phy id %d\n", phy->phy_id);
        return;
    }
}

```

需要根据硬件实际选择的 phy id，在对应的 phy 通道中添加以上 patch 代码。若添加 NO HS-0 patch 后，还是有 MIPI CLK inactive 问题，则需要硬件量信号分析波形和时序。

查看 MIPI CLK 是否有波形出来。

- 若没有波形出来，则硬件分析硬件连接、原理图和 dts 配置是否正确。

- 若有波形出来，初步检测波形的时序是否正常，一般包含 LP11、LP01、LP00、HS-0、HS-00011101 几个阶段的波形。若波形时序不正常，
 - 如在 LP00 波形后有一个异常 HS-1 突起，从而接收端采集到该异常 HS-1 数据，无法正确检测 HS-00011101 同步码，导致 MIPI CLK inactive 问题。该问题可以通过调整 TP2850 的 THS-PREPARE 寄存器，将 LP00 时间调小，把 HS-1 落在接收端的 prepare 时间内，从而忽略掉这个 HS-1 的问题。
 - 如各个阶段波形时间和接收端参数不匹配，接收端无法正确检测到波形时序，无法进入 HS 接收模式，导致 MIPI CLK inactive 问题，需要调整各阶段的波形时间。

说明

- THS-PREPARE + THS-ZERO: $\geq 300\text{ns}$ 。
- THS-PREPARE: $38 \sim 95\text{ns}$ 。
- 分析硬件，比如接线是否错误，MIPI 波形干扰等。